How to escape from local optima in Deep Reinforcement Learning

Hojoon Lee

2022.11.04

Table of Contents

- **Understanding and Preventing Capacity Loss in Reinforcement Learning**
  - ICLR 2022 (spotlight)
  - Clare Lyle, Mark Rowland, and Will Dabney (Oxford & Deepmind)

- **The Primacy Bias in Deep Reinforcement Learning**
  - ICML 2022
  - Evgenii Nikishin, Max Schwarzer (SPR, SGI), D'Oro, Bacon, Aaron Courville (MILA)

Understanding and Preventing Capacity Loss in Reinforcement Learning

Preliminaries

- **DQN with Target Network**
  - Minimize the temporal difference between a predicted Q and the bootstrapped target.

$$\ell_{TD}(Q_\theta, \tau) = (R_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(X_{t+1}, a') - Q_\theta(X_t, A_t))^2$$

- **Training Pipeline**
  - Iterative process of data collection & optimization
    - Data collection: select action with maximum Q-value within an environment.
    - Optimization: minimize TD error.

Understanding and Preventing Capacity Loss in RL

- **Problem**
  - Temporal difference learning is trained to predict non-stationary targets.
  - Due to the non-stationary targets, it is essential for the model to adapt to the new targets.
  - The authors reveal that the networks trained on a sequence of target values lose their ability to quickly update their predictions to new target values (=capacity loss).

- **Solution for Capacity Loss**
  - Present a simple regularization loss, which regress a subspace of features towards its value at initial random model.
  - This simple regularization loss leads to a significant performance improvements in sparse-reward environments such as Montezuma's Revenge.

Measuring Capacity Loss

- **Capacity Loss**
  - A phenomenon which the model losses the ability to fit its prediction to new targets.
  - To validate Capacity Loss, we should first need the measure the Capacity.

Measuring Capacity Loss

- Measure 1: Target-Fitting Capacity

Definition 1 (Target-fitting capacity). *Let $P_X \in \mathscr{P}(X)$ be some distribution over inputs X and $P_{\mathcal{F}}$ a distribution over a family of real-valued functions $\mathcal{F}$ with domain X. Let $\mathcal{N} = (g_\theta, \theta_0)$ represent the pairing of a neural network architecture with some initial parameters θ_0, and $\mathcal{O}$ correspond to an optimization algorithm for supervised learning. We measure the* target-fitting capacity *of $\mathcal{N}$ under the optimizer $\mathcal{O}$ to fit the data-generating distribution $\mathcal{D} = (P_X, P_{\mathcal{F}})$ as follows:*

$$\mathcal{C}(\mathcal{N}, \mathcal{O}, \mathcal{D}) = \mathbb{E}_{f \sim P_{\mathcal{F}}}[\mathbb{E}_{x \sim P_X}[(g_{\theta'}(x) - f(x))^2]] \quad \text{where } \theta' = \mathcal{O}(\theta_0, P_X, f)\,. \tag{3}$$

- measures the ability of a network to reach a new set of targets.
- However, cannot measure how <u>quickly</u> can network update their predictions.

Measuring Capacity Loss

- Measure 2: Feature Rank

Definition 2 (Feature rank). *Let $\phi : X \to \mathbb{R}^d$ be a feature mapping. Let $\mathbf{X}_n \subset X$ be a set of n states in X sampled from some fixed distribution P. Fix $\varepsilon \geq 0$, and let $\phi(\mathbf{X}_n) \in \mathbb{R}^{n \times d}$ denote the matrix whose rows are the feature embeddings of states $x \in \mathbf{X}_n$. Let $\mathrm{SVD}(M)$ denote the multiset of singular values of a matrix M. The feature rank of ϕ given input distribution P is defined to be*

$$\rho(\phi, P, \epsilon) = \lim_{n \to \infty} \mathbb{E}_{\mathbf{X}_n \sim P}[|\{\sigma \in \mathrm{SVD}\left(\frac{1}{\sqrt{n}}\phi(\mathbf{X}_n)\right) | \sigma > \varepsilon\}|] \tag{4}$$

for which a consistent estimator can be constructed as follows, letting $\mathbf{X} \subseteq X, |\mathbf{X}| = n$

$$\hat{\rho}_n(\phi, \mathbf{X}, \epsilon) = |\{\sigma \in \mathrm{SVD}\left(\frac{1}{\sqrt{n}}\phi(\mathbf{X})\right) | \sigma > \varepsilon\}| \,. \tag{5}$$

- Number of singular values of a matrix M = rank of matrix X
- (I think) the authors assume that insufficient rank of matrix X induces a difficulty to adapt to changes in the target values (=capacity loss).

Validating Assumption: Does RL really suffers from Capacity Loss?

- Validation 1: Target-Fitting Capacity of MNIST

and corresponding labels. We first fit a series of labels computed by a randomly initialized neural network f_θ: we transform input-label pairs (x, y) from the canonical MNIST dataset to $(x, f_\theta(x))$, where $f_\theta(x)$ is the network output. To generate a new task, we simply reinitialize the network. Given a target function, we then train the network for a fixed budget from the parameters obtained at the end of the previous iteration, and repeat this procedure of target initialization and training 30 times. We use a subset of MNIST inputs of size 1000 to reduce computational cost. In Figure 1 we

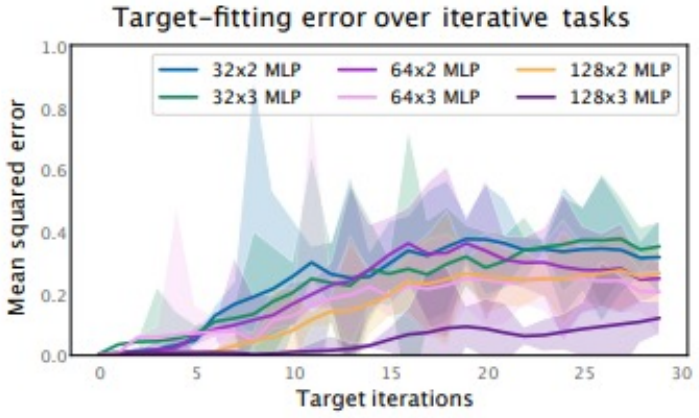


Figure 1: Networks trained to fit a sequence of different targets on MNIST data see increasing error on new target func-

- Error = Target-Fitting Capacity
- Target prediction iteration이 증가함에 따라 Target-fitting Capacity가 감소한다.
- (Why?) (I think the network falls into sharp local optima).
- It is interesting to see that a larger network exhibits a better Target-Fitting Capacity.

Validating Assumption: Does RL really suffers from Capacity Loss?

- Validation 2: Target-Fitting Capacity of Atari

To evaluate **Hypothesis 2**, we train an agent's network checkpoints sampled over the course of training to fit randomly generated target functions. We provide full details of this procedure in Appendix C. We generate target functions by randomly initializing neural networks with new parameters, and use the outputs of these networks as targets for regression. We then load initial parameters from an agent checkpoint at some time t, sample inputs from the replay buffer, and regress on the random target function evaluated on these inputs. We then evaluate the mean squared error after training for fifty thousand steps. We consider a DQN (Mnih et al., 2015), a QR-DQN (Dabney et al.,

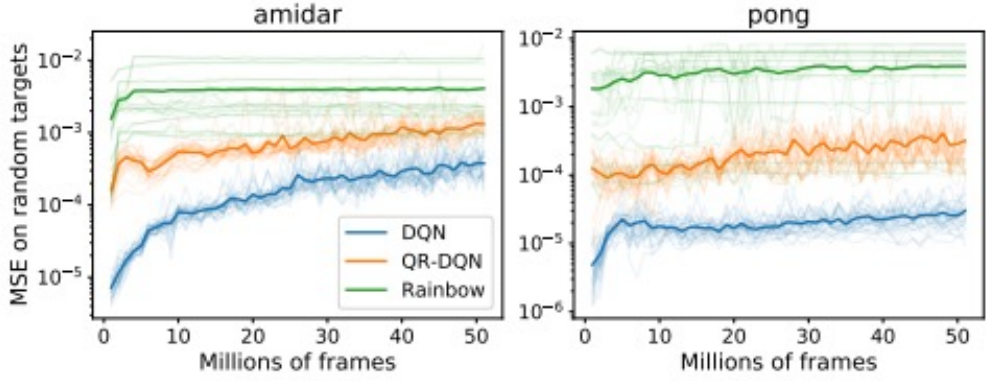


Figure 2: Networks see reduced ability to fit new targets over the course of training in two demonstrative Atari environments.

- As training progresses, agents' checkpoints on average get modestly worse at fitting the random targets.

Validating Assumption: Does RL really suffers from Capacity Loss?

- Validation 3: Feature Rank in Sparse vs Dense reward Environment

In our empirical evaluations, we train a double DQN (DDQN) agent, a quantile regression (QRDQN) agent, and a double DQN agent with an auxiliary random cumulant prediction task (RC DQN) (Dabney et al., 2021), on environments from the Atari suite, then evaluate $\hat{\rho}_n$ with $n = 5000$ on agent checkpoints obtained during training. We consider two illustrative environments: Montezuma's Revenge (sparse reward), and Pong (dense reward), deferring two additional environments to Appendix C.3. We run 3 random seeds on each environment-agent combination.

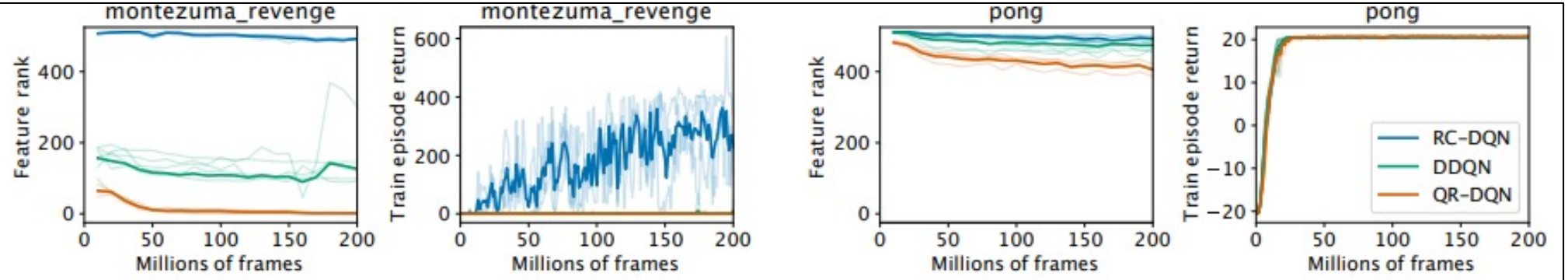
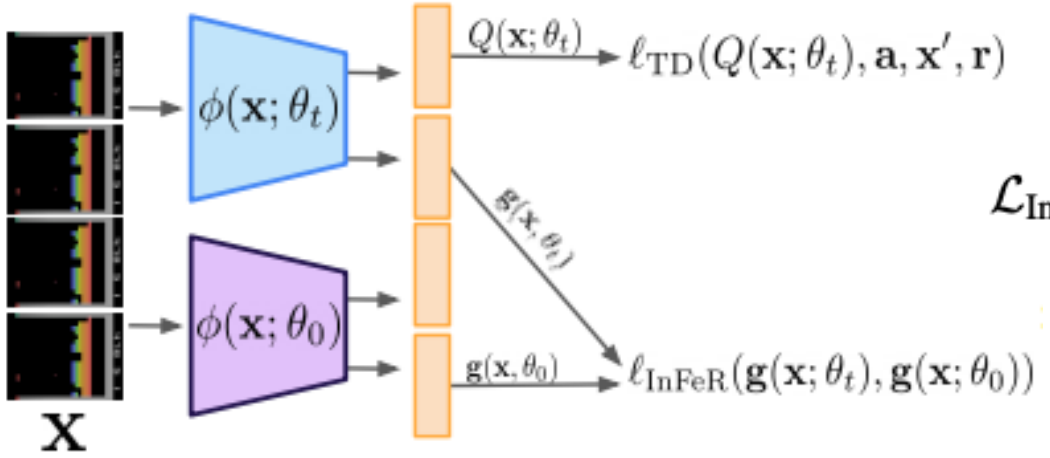


Figure 3: Feature rank and performance over the course of training for Montezuma's Revenge (left) and Pong (right). We observe that feature rank is higher for environments and auxiliary tasks which provide denser reward signals than for sparse reward problems.

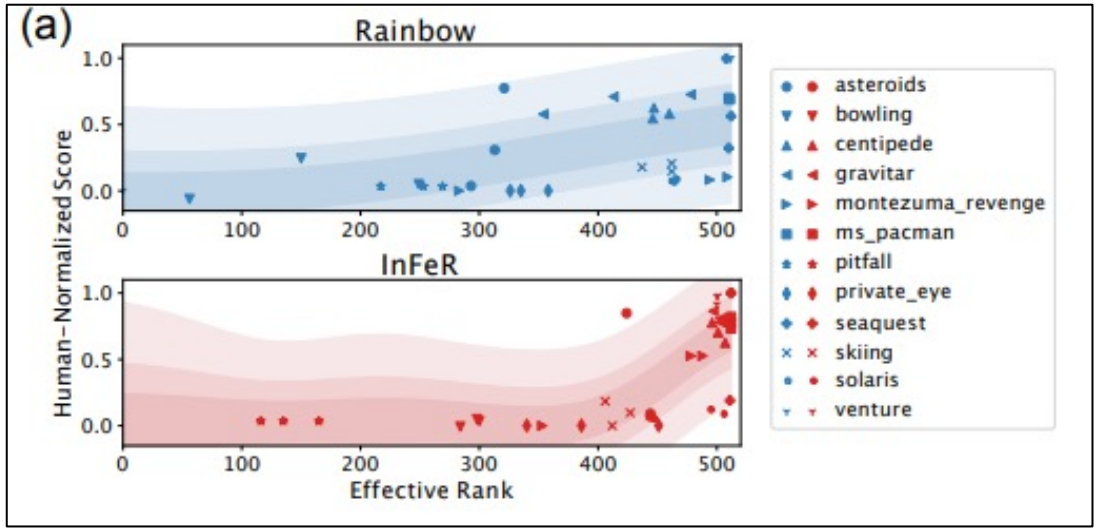
Mitigating Capacity Loss with Feature Regularization.

- **INFER: INitial FEature Regualrization**
  - The authors assume that the randomly initialized network possess high capacity to regress to the new targets.
  - Therefore, add a regularization loss which regress a set of network outputs towards the output values at initialization.
  - In order to not overfit to a specific initialization point, modify the network architecture by adding k auxiliary linear prediction heads on top of the feature representation.

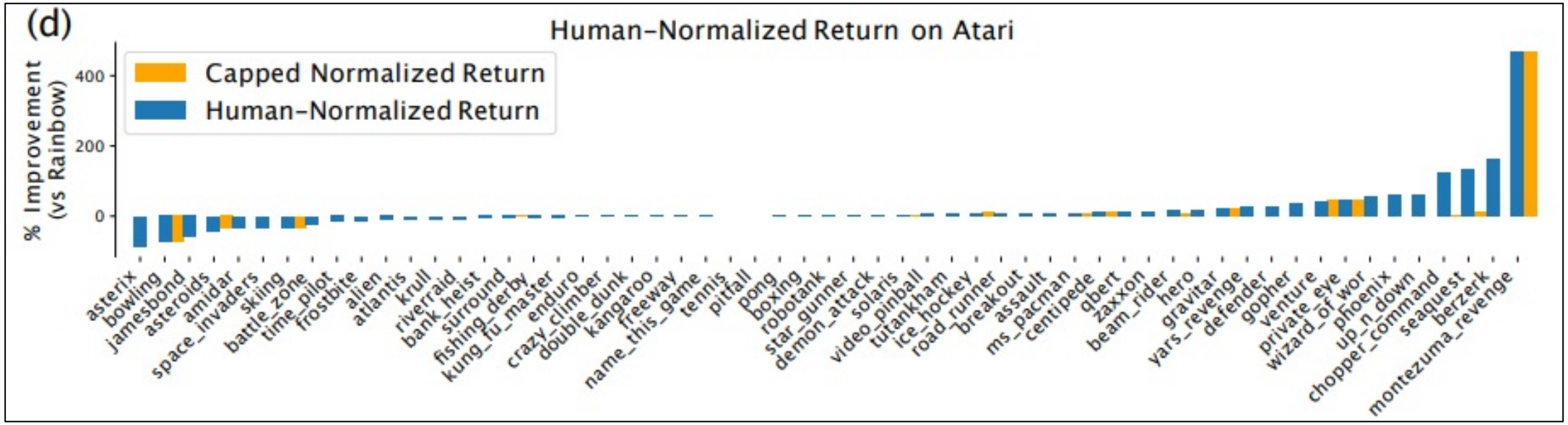


$$\mathcal{L}_{\mathrm{InFeR}}(\theta,\theta_0;\mathcal{B},\beta)=\mathbb{E}_{x\sim\mathcal{B}}\left[\sum_{i=1}^{k}(g_i(x;\theta)-\beta g_i(x;\theta_0))^2\right]$$

Results



- While feature rank does not appear to solely determine agent performance ,there is a positive correlation between FR and HNS.
- Using InFeR increases both FR and HNS.



Conclusion

- **Summary**
  - DRL suffers from Capacity Loss due to the continual update + bootstrapped targets.
  - Capacity Loss is particularly salient in sparse-reward settings.
  - Feature space regularization can help the network to preserve plasticity which increases the agent's performance, especially in sparse-reward environments.

- **Personal thoughts**
  - Capacity Loss == Sharp Local Optima?
    - If the loss falls into sharp local minima, does the model losses the capacity (i.e., plasticity)?
    - Then, can Sharpness-Aware-Minimization or Data Augmentation help to reduce capacity loss?
  - InFeR is the best regularization objective?
    - What about Barlow Twins in SSL? (=Transposed Contrastive Loss)
    - Barlow Twins naturally incur the network's output to be independent.

The Primacy Bias in Deep Reinforcement Learning

The Primacy Bias in Deep Reinforcement Learning

- **Problem**
  - Because deep RL agent is trained on progressively growing datasets, deep RL agents can be easily overfitted to earlier experiences.
  - This overfitting negatively affects the rest of the learning process which the author refer to as primacy bias (중간의 것들보다 처음의 것을 잘 기억하는 인지적 현상).
  - (Personal Thought) Primacy Bias = Capacity Loss = Degenerate Solution

- **Solution for Primacy Bias**
  - Periodically reset a part of the network.

Validating Assumption: Does RL really suffers from Primacy Bias?

- **Validation 1: Heavy Primming.**
  - Investigate whether heavily overfitting on a single batch of early data disrupt a learning process.
    - Environment: DMC quadruped-run.
    - Baseline: SAC.
    - Experiment group: SAC with heavy primming.
      - Before standard training, update the agent 10000 times using the initial 100 data points.

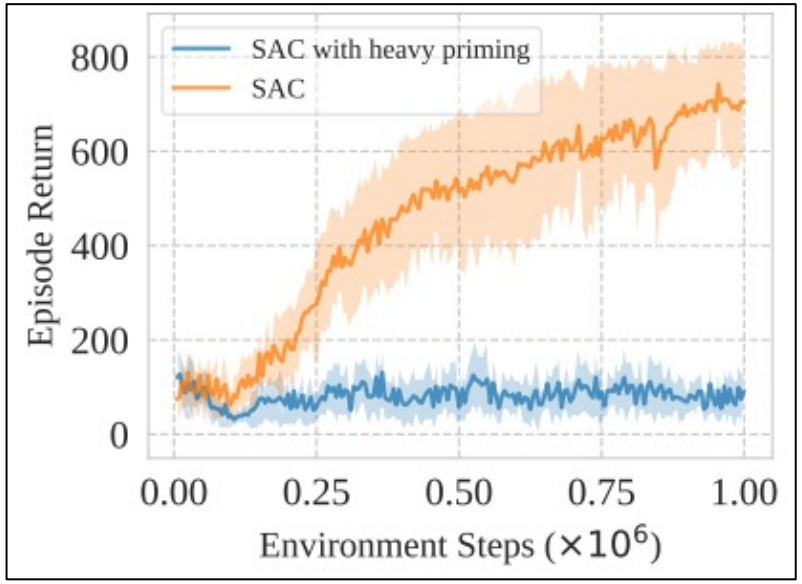


Figure 1. Undiscounted returns on `quadruped-run` for SAC with and without *heavy priming* on the first 100 transitions. An agent extremely affected by the primacy bias is unable to learn even after collecting hundreds of thousands of new transitions. Mean and std are estimated over 10 runs.

Validating Assumption: Does RL really suffers from Primacy Bias?

- **Validation 2: Primacy Bias occurs in data vs model.**
  - Investigate whether non-diverse data incurs the primacy bias.
    1. Generate data with poor performing agent.
    2. Train agent with using collected data as its initial replay buffer.

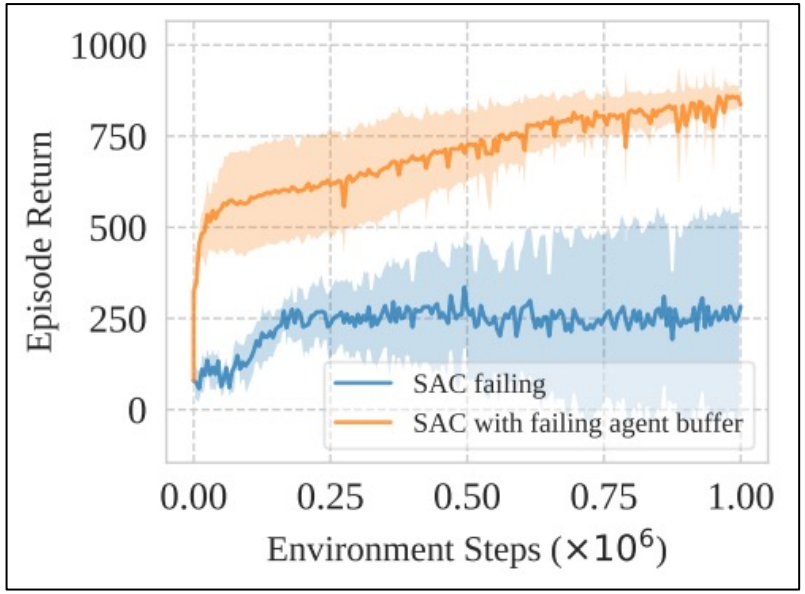


Figure 2. Undiscounted returns on `quadruped-run` for SAC trained with 9 updates per step. `SAC failing` is a standard agent; `SAC with failing agent buffer` is an agent initialized with the replay buffer of the first agent, which allows it to learn quickly. Mean and std are estimated over 10 runs.

  - The primacy bias is a failure to learn not a failure to collect proper data.

Mitigating Primacy Bias with resetting.

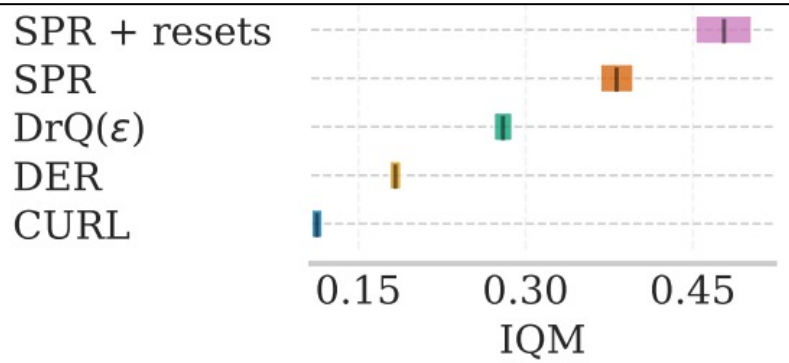
- **Resets**
  - Since the model heavily overfits to the initial data, reset part of the neural network to mitigate the primacy bias in the model.

- **Experimental Setup (ATARI 100k)**
  - Baseline: SPR
  - Number of resets: 4
  - Reset: (1/5)

- **Experimental Setup (DMC)**
  - Baseline: SAC, DrQ
  - Number of resets: 10 (per 200,000 steps)
  - Reset: SAC (all), DrQ (3/7)

Main Experiment

- ATARI 100k



Method	IQM	Median	Mean
SPR + resets	**0.478** (0.46, 0.51)	**0.512** (0.42, 0.57)	**0.911** (0.84, 1.00)
SPR	0.380 (0.36, 0.39)	0.433 (0.38, 0.48)	0.578 (0.56, 0.60)
DrQ(ϵ)	0.280 (0.27, 0.29)	0.304 (0.28, 0.33)	0.465 (0.46, 0.48)
DER	0.183 (0.18, 0.19)	0.191 (0.18, 0.21)	0.351 (0.34, 0.36)
CURL	0.113 (0.11, 0.12)	0.102 (0.09, 0.12)	0.261 (0.25, 0.27)

Figure 3 & Table 1. Point estimates and 95% bootstrap confidence intervals for the performance of SPR with resets and prior methods on Atari 100k. Results for SPR and SPR + resets are over 20 seeds per game; others are taken from Agarwal et al. (2021) and use 100 seeds. The additional baselines contextualize the numerical impact of resets. Per-environment results are available in Appendix C.

- DMC

Method	IQM	Median	Mean
SAC + resets	**656** (549, 753)	**617** (538, 681)	**607** (547, 667)
SAC	501 (389, 609)	475 (407, 563)	484 (420, 548)
DrQ + resets	**762** (704, 815)	**680** (625, 731)	**677** (632, 720)
DrQ	569 (475, 662)	521 (470, 600)	535 (481, 589)

Table 2. Point estimates and 95% bootstrap confidence intervals for the performance of SAC and DrQ with and without resets on DMC tasks. Results are computed over 10 seeds per task. Per-environment learning curves are available in Appendix C.

Ablation: Relationship between replay ratio & reset

- ATARI 100k & DMC

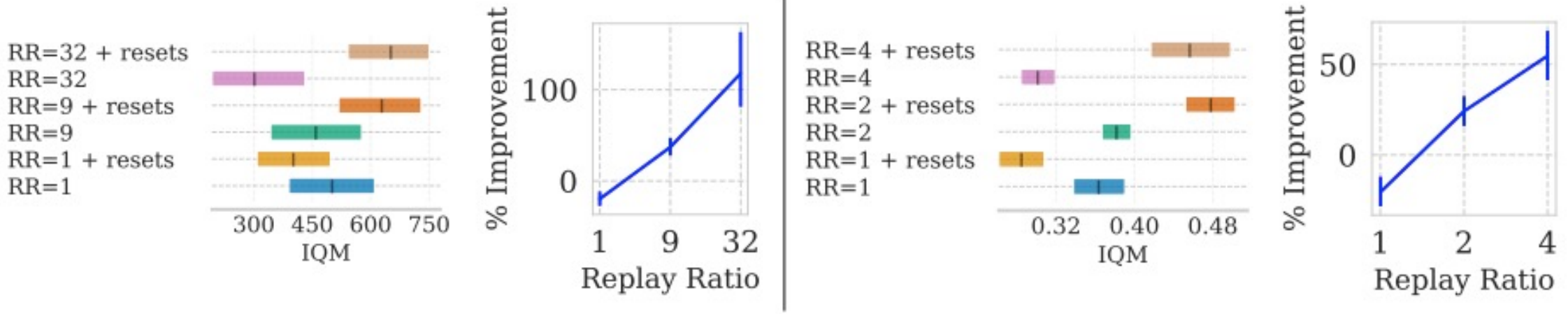


Figure 5. Performance of SAC (left) and SPR (right) and with and without resets for different replay ratios and a fixed default n. The right-hand plots visualize the percent improvement gained by adding resets. Agents with higher replay ratio are more prone to the primacy bias and hence benefit more from mitigating it.

Ablation: where to reset?

- ATARI 100k

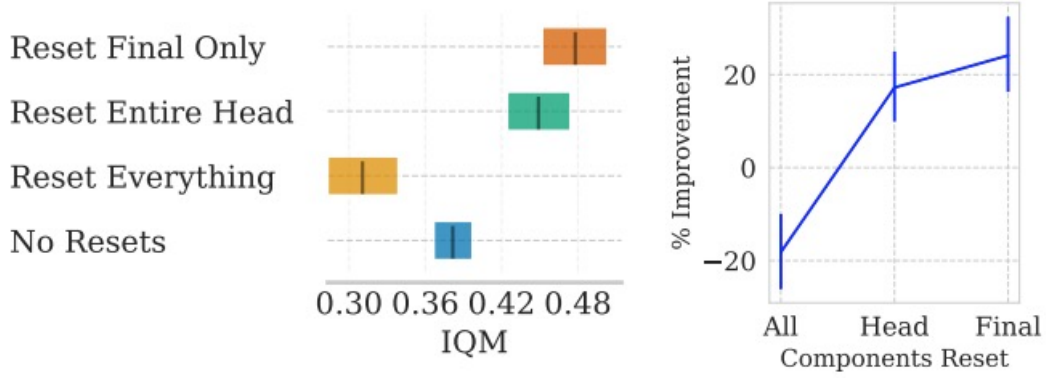


Figure 13. Performance of SPR when resetting different groups of parameters. The right plot visualizes the percentage of improvement gained by resetting a certain group of parameters compared to no resets at all. Resetting the last layer only delivers a slightly higher IQM than resetting the 2-layer head, while resetting the whole network noticeably damages the performance.

Ablation: where to reset?

- DMC

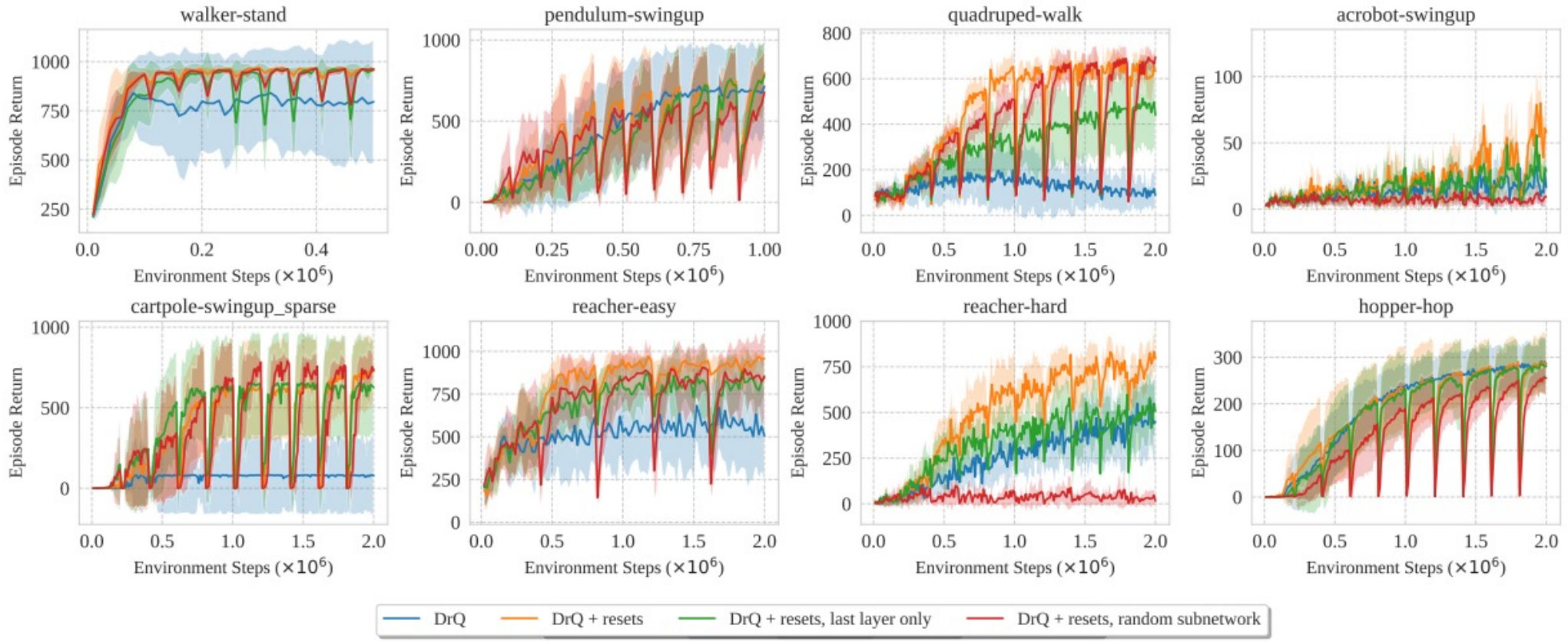


Figure 10. Performance of DrQ with resetting the 3-layer heads (i.e. standard resets), only the last layers, and random subnetworks. The overall performance of the latter two versions is either comparable or worse.

Ablation: what to reset?

- DMC

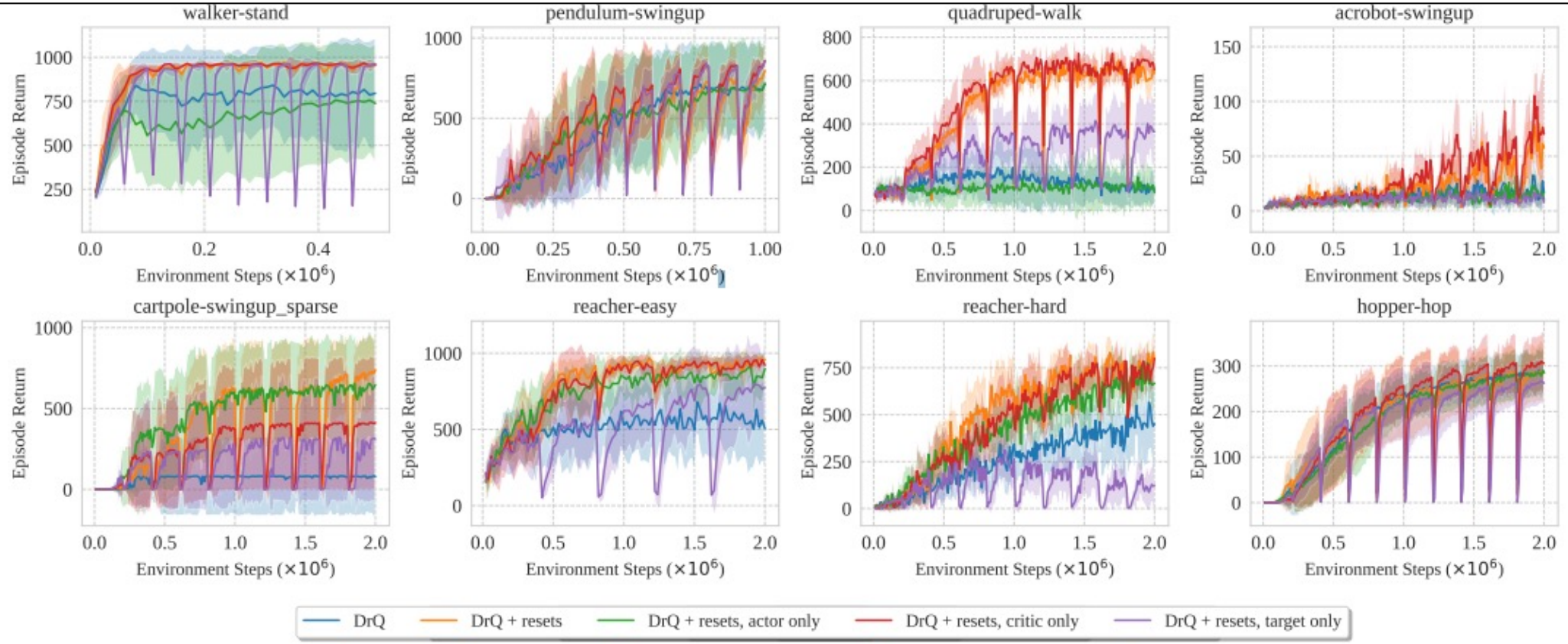


Figure 11. Performance of DrQ with resets of actor, critic, and target critic networks simultaneously (i.e. standard resets) and individually. Resetting the critic yields the predominant effect in most environments, but resetting all networks proved to be the most robust option.

Conclusion

- **Summary**
  - DRL suffers from to adapt to new data due to the Primacy Bias.
  - Resetting part of the neural network helps to alleviate Primacy Bias issue.

- **Personal thoughts**
  - Capacity Loss == Primacy Bias == Local Optima?
  - Plausible Solutions: (w/ a larger replay ratio).
    - Regularization Loss: InFeR or Barlow Twins.
    - Random network resets.
    - Data augmentation (augmentation leads to escape sharp local minima).
      - Is this the reason why augmentation provide a larger benefits in deep RL?
    - Sharpness-aware optimization.

Q & A